RV COLLEGE OF ENGINEERING® BENGALURU-59

(Autonomous Institution Affiliated to VTU, Belagavi)

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Certificate

Certified that the Operating Systems Lab Experiential Learning titled ***"Sandboxed Program Runner [using ptrace and seccomp]"*** is carried out by in partial fulfilment for the requirement of degree of **Bachelor of Engineering in Computer Science and Engineering** of the Visvesvaraya Technological University, Belagavi during the year 2025-2026. It is certified that all corrections/suggestions indicated for the Internal Assessment have been incorporated in the report.

Prof. Rajatha,	**Dr. Shanta Rangaswamy,**
Assistant Professor,	**Professor and HoD,**
Department of CSE	**Department of CSE**

External Viva

Name of Examiners	Signature with Date
1.	
2.	

Rashtreeya Sikshana Samithi Trust

RV COLLEGE OF ENGINEERING®

(Autonomous Institution Affiliated to VTU, Belagavi)

Operating Systems Experiential Learning Report

Topic: Sandboxed Program Runner [Using ptrace and seccomp]

By:
Praneel K A (1RV24CS200)
Rohan S(1RV24CS232)

Prof. Rajatha

TABLE OF CONTENTS:

INTRODUCTION**4**
PROBLEM STATEMENT4
OBJECTIVES4
METHODOLOGY6

1. System Architecture Overview6
2. Backend development phase
3. Feature Intergration phase
4. Frontend development phase
5. Testing and Optimization phase
7. Architecture design
8. Sandbox Functions
9.Operating System Concepts Used

EXPEXTED OUTCOMES
CONCLUSION16
REFERENCES16

INTRODUCTION

The Linux Sandbox is a security-focused program execution environment designed to safely run and analyze untrusted executables. It combines low-level system programming in C (for system call tracing and risk analysis) with a flask backend API and a modern frontend interface for visualization.

At its core, the project uses *ptrace-based system call interception*, process isolation techniques, and a rule-based scoring engine to detect potentially malicious behavior. The system is capable of logging every syscall invoked by a running executable, classifying its behavior, and generating a real-time risk report.

The key OS-level implementation and full-stack pipeline:

- C Backend (Core Sandbox) – controls program execution, monitors syscalls, computes risk scores
- Flask Backend Server – exposes REST APIs, stores logs, manages execution requests
- Frontend Web UI – allows uploading executables, running tests, and viewing syscall traces & risk levels
- Malicious Test Samples – included to benchmark and validate detection accuracy

This cross-layer architecture makes the project a powerful demonstration of Operating Systems concepts, security engineering.

PROBLEM STATEMENT

Running untrusted programs directly on a user's system poses significant risks:

- Malware can execute harmful system calls
- Filesystem or network-sensitive operations may occur unnoticed
- Programs may escalate privileges or attempt persistence
- No simple way exists to *observe system calls in real time*

- Non-technical users cannot understand the behavior of unknown executables
- Traditional sandboxing solutions are complex and difficult for beginners to use

Current Challenges

Existing sandboxes (Firejail, nsjail, AppArmor) are powerful but:

- Hard to configure
- Not beginner-friendly
- Provide no visual analysis dashboard
- Require command-line knowledge
- Do not offer risk scoring or easy-to-understand classification

Proposed Solution

The Linux Sandbox system addresses these limitations through:

- ptrace-based syscall tracing
- Rule-based malicious behavior scoring
- Safe execution in a controlled environment
- Backend APIs for automation
- Frontend dashboard for visualization
- Integration of test malicious files for validation

Together, these features create an accessible, educational, and practical OS-level security tool.

OBJECTIVES

The primary objectives are:

1. Implement a secure sandbox that executes untrusted binaries using fork, execve, ptrace, and waitpid.
2. Intercept every system call invoked by the target program.
3. Build a behavior scoring system that identifies risky operations.
4. Create a backend REST API to manage execution requests and logs.
5. Develop a frontend dashboard to visualize:
    - Syscall traces
    - Risk score
    - Verdict (Safe / Suspicious / Malicious)
6. Include malicious sample programs to benchmark detection capability.
7. Demonstrate fundamental OS concepts:

- Process creation
- System calls
- Kernel-user mode transitions
- Memory management
- IPC

8. Achieve a clean, modular, and extensible architecture.

METHODOLOGY

System Architecture Overview

Component	Description
C Sandbox Core	Executes target program under ptrace, logs syscalls, assigns risk scores
Backend API (Flask)	Handles file uploads, runs sandbox executable, stores logs
Frontend UI	Displays syscall logs, risk scores, and final verdict
Testing Samples	Malicious executables for evaluating detection accuracy

The architecture follows:

Frontend → Flask Backend → Sandbox (C Program) → System Calls → Output → Flask → Frontend

Backend Development Phase

The backend of the system is implemented using Python's Flask framework, which acts as the central communication layer between the frontend UI and the C-based sandbox engine.

Backend Responsibilities:

- Providing REST API endpoints for uploading executables
- Invoking the sandbox C program with uploaded files
- Reading the sandbox-generated output (system call log, risk score, verdict)
- Returning results as JSON responses to the frontend
- Handling file storage, temporary directories, and cleanup
- Logging sandbox activity for further analysis

Key Backend Components:

- Flask Web Server – Light-weight, Python-based backend for handling API requests
- Flask-Uploads – For safe file upload handling
- subprocess module – To execute the sandbox C program with proper argument passing
- JSON response handlers – To send structured logs, scores, and verdict to the frontend
- Local file storage – For temporarily saving uploaded binaries and sandbox output

Backend Workflow:

- User uploads an executable from the frontend
- Flask receives the file via a POST request
- Flask stores the file in a secure temporary directory
- Flask executes the sandbox using:
    - subprocess.Popen(["./sandbox", uploaded_file_path], ...)
- Flask reads:
    - Syscall traces
    - Risk score
    - Classification verdict
- Flask formats results into JSON and sends them to the frontend
- Temporary files are cleaned after processing

Feature Integration Phase

Major integrations:

- Ptrace syscall interception :- Forward logs to backend
- Risk scoring rules (e.g., suspicious syscalls: openat, execve, ptrace, network calls)
- Verdict classification (Safe / Suspicious / Malicious)
- API response formatting
- Frontend visualization parsing

Frontend Development Phase

The frontend of the sandbox system is implemented using React, bootstrapped with Create React App, as indicated by the project's configuration files

It allows users to upload executables, trigger sandbox execution, and visualize syscall logs and the final risk verdict.

Key Responsibilities of the Frontend

- Provide a file upload UI for selecting executables
- Send uploaded files to the backend Flask API
- Display sandbox results:
  - Syscall logs
  - Risk score
  - Verdict (Safe / Suspicious / Malicious)
- Show loading states while the backend processes the program
- Render all results dynamically using React state updates

Technology Stack

The frontend is built using:

- React 19 (as per react and react-dom in package.json)
  - package
- react-scripts for build & development environment
- JavaScript (ES6+)
- Fetch API / Axios (depending on the code) to send requests to Flask
- HTML5 File Input for selecting binaries
- React Hooks (useState, useEffect) for managing UI states

Testing and Optimization Phase

- Multiple malicious samples were tested:
- File manipulation malware
- Suspicious syscall patterns
- Reverse shell attempt dummy executables
- Abnormal memory usage programs
- The sandbox behavior was validated for:
- Correct syscall interception
- Accurate scoring

- Stable frontend rendering
- Smooth backend execution
- No OS crashes or unsafe operations

Architecture Design

1. System Components Overview

A. Sandbox Engine (C Program)

The core sandbox is implemented as a modular C codebase, divided into .c files and corresponding header files, all managed through a single Makefile. This structure keeps the system organized, readable, and easy to extend. Instead of writing one large monolithic C program, each major functionality is separated into its own module:

- main.c – starts the sandbox, forks the process, initializes risk manager
- child_process.c – handles resource limits, applies seccomp, executes target program
- monitor.c – the ptrace-based monitoring loop that inspects every syscall
- syscall_utils.c – maps syscall numbers to names and reads strings from child memory
- risk_manager.c – maintains risk score, mmap count, and risk evaluation logic
- path_utils.c – checks whether accessed file paths are sensitive
- seccomp_filter.c – constructs and loads the seccomp BPF allowlist filter

Each .c file has a matching .h header file, which provides:

- Function declarations
- Shared data structures
- Cross-module interfaces

This modular separation improves maintainability and ensures each component has a single responsibility, making the system easy to debug and upgrade.

Finally, a Makefile compiles all modules together, handles dependencies, and produces the sandbox executable. This also ensures consistent builds across different environments.

B. Backend API (Flask Framework)

- Accepts file uploads from the frontend
- Runs the C sandbox using Python's subprocess
- Reads sandbox output and converts it to JSON
- Sends analysis results back to the frontend

Flask acts as the communication bridge and ensures the frontend never interacts with the system directly.

C. Frontend Interface (React App)

- Provides a simple UI to upload executables
- Sends files to Flask via REST API
- Displays syscall logs, risk score, and verdict
- Uses React state to update results dynamically

The frontend focuses entirely on ease of use and clear visualization.

2. Data Flow Through the System

1. User uploads executable via React UI
2. Flask stores the file and launches the sandbox
3. Sandbox analyzes the program, logs system calls, and computes the verdict
4. Flask reads results and sends them as JSON
5. React renders the output (syscalls, score, classification)

This structured pipeline ensures isolation, safety, and clarity.

3. Benefits of the Architecture

- Security: Untrusted programs run only inside the sandbox
- Modularity: Frontend, backend, and sandbox are independent
- Extensibility: Easy to add features like charts, log history, or more analysis rules
- Maintainability: Each layer has a clear, single responsibility

Sandbox Functions

The sandbox system internally uses lightweight but effective data structures and algorithms to track program behavior, monitor system calls, and compute malicious intent.

1. Risk Scoring System

The risk engine is built around simple integer counters:

- int risk_score — global score representing total malicious activity
- int mmap_count — tracks number of memory mappings performed by the program
    - Associated helper functions:
- add_risk(points)
- increment_mmap_count()

- get_risk_score()
- get_mmap_count()

This design keeps the scoring mechanism fast, deterministic, and easy to tune.

2. System Call Classification

The sandbox uses a mapping where the lookup table efficiently converts numeric syscall IDs into readable names.

3. String Extraction From Child Process

The function read_child_string() uses:

- ptrace(PTRACE_PEEKDATA)
- manual memory word copying
- termination detection via memchr()
    - Algorithm characteristics:
- Reads memory in 8-byte chunks
- Stops when null terminator is found
- Prevents buffer overflow by capping at 255 bytes

This allows the sandbox to inspect file paths passed to openat().

4. Sensitive Path Detection

The function is_sensitive() uses substring search via strstr():

strstr(path, "/etc/passwd")

strstr(path, "/etc/shadow")

- Used only for short fixed patterns → effectively constant time

5. Seccomp BPF Filter (Berkeley Packet Filter Program)

seccomp_filter.c constructs a BPF instruction array that:

- Loads syscall number

- Compares against a strict allowlist
- Blocks everything else via SECCOMP_RET_TRAP
    - Algorithm type:
- Static filtering using linear BPF rule evaluation
- Extremely fast, enforced in kernel space
    - This ensures only safe syscalls are allowed to execute.

6. Event-Driven Trace Loop

Inside monitor.c, the monitor() function uses:

- ptrace event loop
- WIFSTOPPED / WIFSIGNALED / WIFEXITED state machine
- Registers extraction via PTRACE_GETREGS
    - Algorithm pattern:
- State-machine-based event processing
- Each syscall event triggers risk evaluation
- Detects termination, crashes, and seccomp violations

Operating System Concepts

The sandbox leverages several fundamental and advanced OS concepts. Together, they form the backbone of how modern debugging tools, malware analysis systems, and container sandboxes operate. The project demonstrates how low-level OS mechanisms can be combined to build a secure execution environment.

1. Process Creation and Management

The sandbox uses fork() to create a new child process that executes the untrusted binary. This demonstrates:

- How a process duplicates its execution context
- How parent and child maintain independent memory spaces
- How parents manage child state through waitpid()
- Key lifecycle states: running → stopped → terminated

This reflects how real OS process schedulers and supervisors manage user programs.

2. Program Loading and Replacement (execve)

The child process uses execvp() to replace its memory with the untrusted program. This illustrates:

- Loading a new executable image into memory
- Clearing the previous stack, heap, and code sections
- Passing command-line arguments to the new program
- Starting execution from main() of the target binary

It shows how the OS performs *program replacement*, not just program execution.

3. System Call Interface & Kernel Boundary

Every operation performed by the traced program—file access, memory allocation, process termination—occurs through system calls.

The sandbox intercepts syscalls such as: openat, pread64, mmap, close, execve, brk

This demonstrates:

- How user programs request OS services
- How the CPU transitions between user mode and kernel mode
- The division of privilege levels in modern Operating Systems

Monitoring syscalls gives insight into the behaviour of a process at its most fundamental level.

4. ptrace-Based System Call Tracing

The core of the sandbox uses ptrace()

Concepts involved:

- Child declares itself traceable with PTRACE_TRACEME
- Parent traps syscall-entry and syscall-exit
- Registers (like orig_rax) hold syscall numbers
- The tracer (sandbox) can inspect memory using PTRACE_PEEKDATA

This exposes low-level introspection abilities that normal programs cannot use.

5. Seccomp Filtering (Kernel-Level Sandboxing)

The code uses seccomp in filter mode, which applies a BPF (Berkeley Packet Filter) program directly in the kernel. It limits the child to a strict set of allowed syscalls.

OS concepts demonstrated:

- Mandatory Access Control (MAC)
- Kernel-enforced syscall allowlists
- SECCOMP_RET_TRAP for forbidden calls

- Privilege restriction via PR_SET_NO_NEW_PRIVS

6. Signals and Interrupt Handling

The monitor loop handles several OS signals:

- SIGTRAP - indicates syscall interception
- SIGSYS - seccomp violation
- SIGXCPU - exceeded CPU time limit
- SIGKILL - abnormal process termination

Concepts shown:

- How the OS uses signals for asynchronous communication
- How processes respond to exceptions and faults
- How system call traps are forwarded to the tracer

This gives the sandbox control over abnormal or malicious behaviour.

7. File System Access and Directory Permissions

System calls such as: openat, pread64, lseek, fstat allow the OS to manage:

- How permissions (read/write/execute) are applied
- File descriptors
- Permission checks
- Path resolution
- Interaction with /etc, /proc, /sys

The sandbox even detects access to sensitive files (e.g., /etc/passwd, /etc/shadow) using path inspection.

8. Resource Limits (rlimit)

The child process is constrained using POSIX resource limits:

- RLIMIT_CPU → restricts CPU time
- RLIMIT_AS → restricts virtual memory

Concepts demonstrated:

- How OS prevents runaway processes
- Enforcing memory ceilings to stop overflow attacks

9. Process State Inspection & Wait Mechanisms

The monitor uses waitpid() with macros such as:

- WIFEXITED
- WIFSIGNALED
- WIFSTOPPED
- WSTOPSIG

These enable the tracer to understand:

- Whether the child exited normally
- Whether it was killed by a signal
- Whether it is stopped due to ptrace events

10. Kernel-Level BPF (Berkeley Packet Filter) Execution

The seccomp rules compile into BPF instructions evaluated inside the kernel.
Concepts include:

- Register loading
- Conditional jumps
- Filter acceptance or rejection
- Kernel-level packet filtering logic adapted for syscalls

This is a powerful OS mechanism repurposed for security.

EXPECTED OUTCOMES

The sandbox system is expected to provide a secure and reliable environment for analyzing untrusted executables. The main outcomes are:

1. Accurate System Call Monitoring

- Every syscall made by the program is captured and displayed.
- Sensitive actions (file access, memory operations) are clearly visible.

2. Reliable Malicious Behaviour Detection

- Low-risk programs produce **SAFE** verdicts.
- Suspicious patterns (repeated mmap, unusual reads/writes) increase the risk score.
- Seccomp violations, sensitive file access, or abnormal termination lead to MALICIOUS verdicts.

3. Safe Execution Environment

- CPU time and memory limits prevent resource abuse.
- Dangerous syscalls are blocked by seccomp filters.
- The program cannot modify system files or escape the sandbox.

4. Clear Output and Verdict

- The system prints a final **risk score** and classification (SAFE / SUSPICIOUS / MALICIOUS).
- Verdicts are consistent across repeated executions.

5. Functional Frontend–Backend Workflow

- Users can upload a binary through the frontend UI.
- Flask backend executes the sandbox and returns results.
- The UI displays syscall logs, scores, and verdict cleanly.

CONCLUSION

The developed sandbox system successfully demonstrates how core Operating System concepts—such as system calls, process control, ptrace monitoring, seccomp filtering, and resource management—can be combined to create a secure execution environment for untrusted programs. By intercepting and analyzing every syscall, enforcing strict kernel-level restrictions, and applying a rule-based risk evaluation model, the system is able to differentiate between safe, suspicious, and malicious behaviour effectively.

The integration of a Flask backend and a React-based frontend also provides a smooth and user-friendly interface, making the analysis process accessible even to non-technical users. Overall, the project meets its objectives by delivering a lightweight, practical, and educational sandbox that mirrors real-world malware analysis and process monitoring tools while reinforcing important OS principles in a hands-on manner.

REFERENCES

[1] Wan, Z., Lo, D., Xia, X., & Cai, L. (2019). Practical and effective sandboxing for Linux containers. *Empirical Software Engineering*, 24(6), 3396–3442

[2] Haq, M. S. U., Liao, L., & Lerong, M. (2016). Design and implementation of sandbox technique for isolated applications. In *2016 IEEE Information Technology, Networking, Electronic and Automation Control Conference* (pp. 774–779)

[3] Garfinkel, T., Pfaff, B., & Rosenblum, M. (2004). Ostia: A delegating architecture for secure system call interposition. In *Proceedings of the Network and Distributed System Security Symposium* (NDSS).

[4] Provos, N. (2003). Improving host security with system call policies. In *Proceedings of the 12th USENIX Security Symposium* (pp. 257–272). USENIX Association. [5] Tanenbaum, A. S. (2022). *Modern Operating Systems* (5th ed.). Pearson Education.